Creating Maps and Visualizing Geospatial Data

What I'm Doing

In this lab, I explore how to create interactive maps and visualize geospatial data using the Folium library in Python. I'll work through creating basic world maps, customizing map styles, plotting incident markers, and building a choropleth map to visualize immigration patterns to Canada.

1. Setting Up

First, I install and import the necessary libraries. The core dependencies are NumPy and pandas for data handling, and Folium for map-making. Folium doesn't come pre-installed with most Python environments, so I install it first:

```
import numpy as np
import pandas as pd
#%pip install folium
import folium
```

2. Getting to Know the Dataset

This lab works with two datasets:

1. **San Francisco Police Department Incidents (2016)** - over 150,000 crime records with location coordinates.
2. **Canadian Immigration Data (1980–2013)** - annual migration flows to Canada from countries worldwide.

Let me **load the San Francisco incidents dataset** first:

```
df_incidents = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/
Data%20Files/Police_Department_Incidents_-_Previous_Year__2016_.csv')
```

Let me inspect the first few rows to understand the structure:

```
df_incidents.head()
```

	IncidntNum	Category	Descript	DayOfWeek	Date	Time	PdDistrict	Resolution
0	120058272	WEAPON LAWS	POSS OF PROHIBITED WEAPON	Friday	01/29/2016 12:00:00 AM	11:00	SOUTHERN	ARREST, BOOKED
1	120058272	WEAPON LAWS	FIREARM, LOADED, IN VEHICLE, POSSESSION OR USE	Friday	01/29/2016 12:00:00 AM	11:00	SOUTHERN	ARREST, BOOKED
2	141059263	WARRANTS	WARRANT ARREST	Monday	04/25/2016 12:00:00 AM	14:59	BAYVIEW	ARREST, BOOKED
3	160013662	NON-CRIMINAL	LOST PROPERTY	Tuesday	01/05/2016 12:00:00 AM	23:50	TENDERLOIN	NONE
4	160002740	NON-CRIMINAL	LOST PROPERTY	Friday	01/01/2016 12:00:00 AM	00:30	MISSION	NONE

Each row contains **13 features**: an incident number, crime category, description, day of week, date, time, police district, resolution status, address, longitude (X), latitude (Y), a location tuple, and a police department ID.

Let me check the total number of records:

```
df_incidents.shape
```

```
(150500, 13)
```

Result: Over 150,000 recorded incidents in 2016.

3. Cleaning the Data

Rendering 150,000 markers on a map would be computationally expensive, so I trim the dataset to the **first 100 incidents** for this demonstration:

```
limit = 100
df_incidents = df_incidents.iloc[0:limit, :]
df_incidents.shape
```

```
(100, 13)
```

Result: 100 incidents, all 13 columns preserved.

Now I also **load the Canadian immigration dataset** that I'll use later for the choropleth map:

```
df_can = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-
```

```
storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/
Data%20Files/Canada.csv')

df_can.head()
```

	Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	...	2005	2
0	Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	...	3436	
1	Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	...	1223	
2	Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	...	3626	
3	American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	...	0	
4	Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	...	0	

5 rows × 39 columns

Result: A table showing the first five rows, with columns for Country, Continent, Region, Developing status, individual year columns (1980–2013), and a Total column summing all years.

```
print(df_can.shape)
```

```
(195, 39)
```

Result: 195 countries with 39 columns (country metadata + annual immigration figures).

4. Introduction to Folium

Folium is a Python library built on the Leaflet.js mapping library. Its key advantage is that it produces **interactive** maps - you can pan, zoom, and click on elements - and it's **completely free** with no API call limits.

4.1 Creating a Basic World Map

Generating a world map is straightforward:

```
world_map = folium.Map() # define the world map
world_map # display world map
```

Result: An interactive world map rendered in the notebook, centered by **default at [0, 0]** with a global view.

4.2 Centering and Zoom

I can customize the map by specifying a center point **(latitude, longitude)** and an **initial zoom level**:

```
world_map = folium.Map(location=[56.130, -106.35], zoom_start=4)
world_map
```

Result: A world map **centered on Canada at zoom level 4** - you see the whole country and

surrounding regions.

At a **higher zoom level**, the map **zooms in** further:

```
world_map = folium.Map(location=[56.130, -106.35], zoom_start=8)
world_map
```

Result: The map zooms in closer on Canada, showing more geographic detail.

Exercise: Create a Map of Mexico

Create a **map centered on Mexico** with a **zoom level of 4**.

```
mexico_latitude = 34.307144
mexico_longitude = -106.018066

mexico_map = folium.Map(location=[mexico_latitude, mexico_longitude],
zoom_start=4)
mexico_map
```

Result: An interactive map centered on Mexico at zoom level 4, showing the **country in full view.**

4.3 Changing Map Styles

Folium supports different tile styles. One useful option is **Cartodb dark_matter** - high-contrast **black-and-white maps**, great for overlaying data and exploring river meanders and coastal zones:

```
# map of the world centered around Canada
world_map = folium.Map(location=[56.130, -106.35], zoom_start=4, tiles='Cartodb
dark_matter')
world_map
```

Result: A clean, **dark-colored map** of Canada with visible borders and English country names.

Exercise: Create a Cartodb Positron Map of Mexico

Create a map of Mexico using the Cartodb positron tiles with a zoom level of 6.

```
mexico_latitude = 23.6345
mexico_longitude = -102.5528

mexico_map = folium.Map(location=[mexico_latitude, mexico_longitude],
zoom_start=6, tiles='Cartodb positron')
mexico_map
```

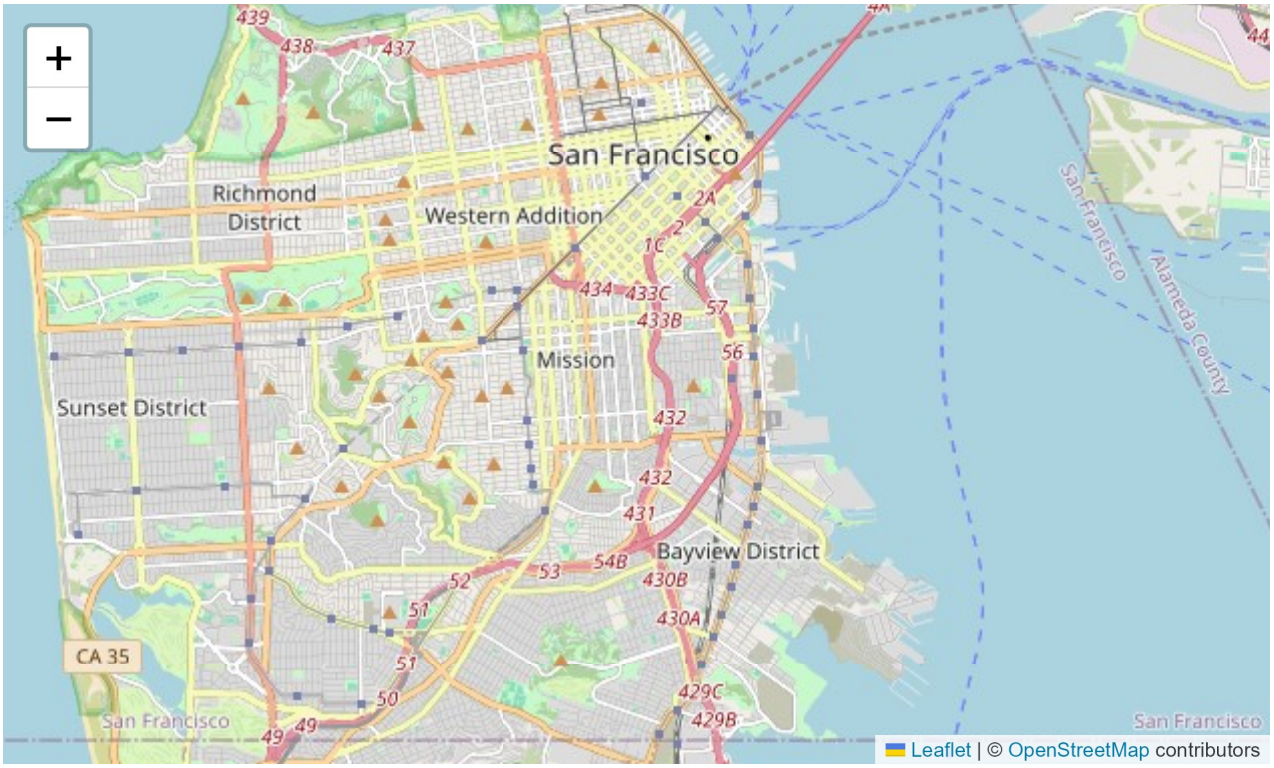
Result: A clean, modern, light-themed map centered on Mexico at zoom level 6.

5. Maps with Markers

Now I'll **plot the 100 San Francisco crime incidents on a map** to visualize their locations.

5.1 Base Map of San Francisco

```
latitude = 37.77
longitude = -122.42

sanfran_map = folium.Map(location=[latitude, longitude], zoom_start=12)
sanfran_map
```

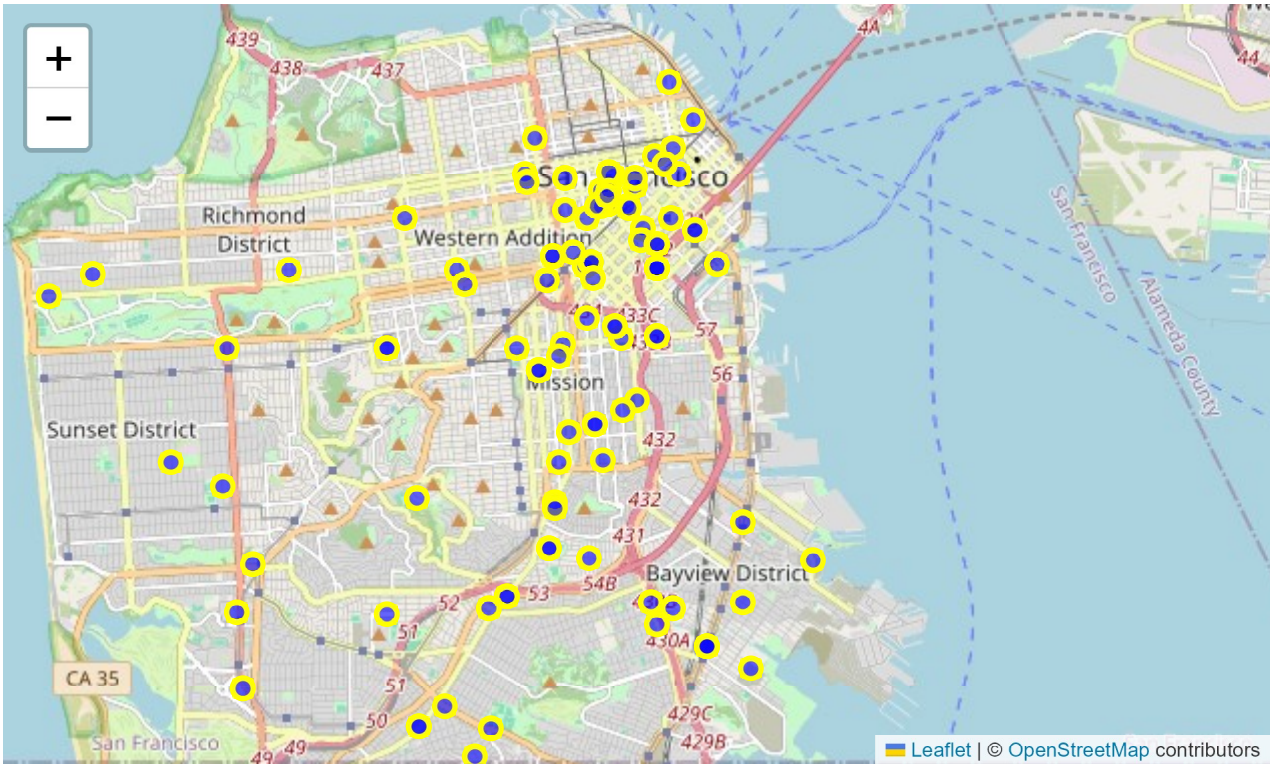


Result: An interactive map of San Francisco at a **street-level zoom**.

5.2 Adding Circle Markers

I create a feature group and loop through the incident coordinates, adding a blue-filled circle marker for each crime:

```
incidents = folium.map.FeatureGroup()

# loop through the 100 crimes and add each to the incidents feature group
for lat, lng in zip(df_incidents.Y, df_incidents.X):
    incidents.add_child(
        folium.vector_layers.CircleMarker(
            [lat, lng],
            radius=5, # marker size
            color='yellow',
            fill=True,
            fill_color='blue',
            fill_opacity=0.6
        )
    )
# add incidents to map
sanfran_map.add_child(incidents)
```

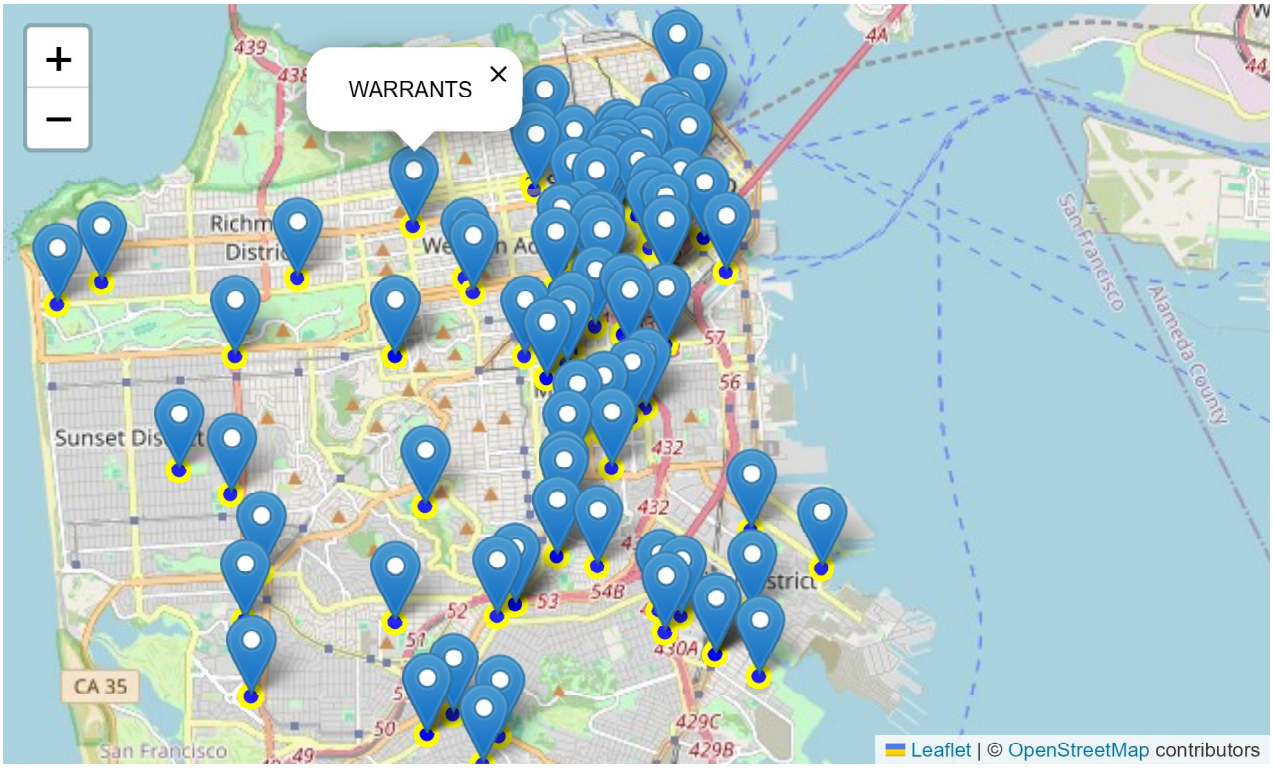


Result: The San Francisco map now displays 100 blue circle markers, **one for each crime** incident location.

5.3 Adding Popup Text

I can make the map interactive by adding popup labels that display the crime category when clicked:

```
sanfran_map = folium.Map(location=[latitude, longitude], zoom_start=12)

incidents = folium.map.FeatureGroup()

for lat, lng in zip(df_incidents.Y, df_incidents.X):
    incidents.add_child(
        folium.vector_layers.CircleMarker(
            [lat, lng],
            radius=5,
            color='yellow',
            fill=True,
            fill_color='blue',
            fill_opacity=0.6
        )
    )
# add pop-up text to each marker on the map
latitudes = list(df_incidents.Y)
longitudes = list(df_incidents.X)
labels = list(df_incidents.Category)

for lat, lng, label in zip(latitudes, longitudes, labels):
    folium.Marker([lat, lng], popup=label).add_to(sanfran_map)

sanfran_map.add_child(incidents)
```

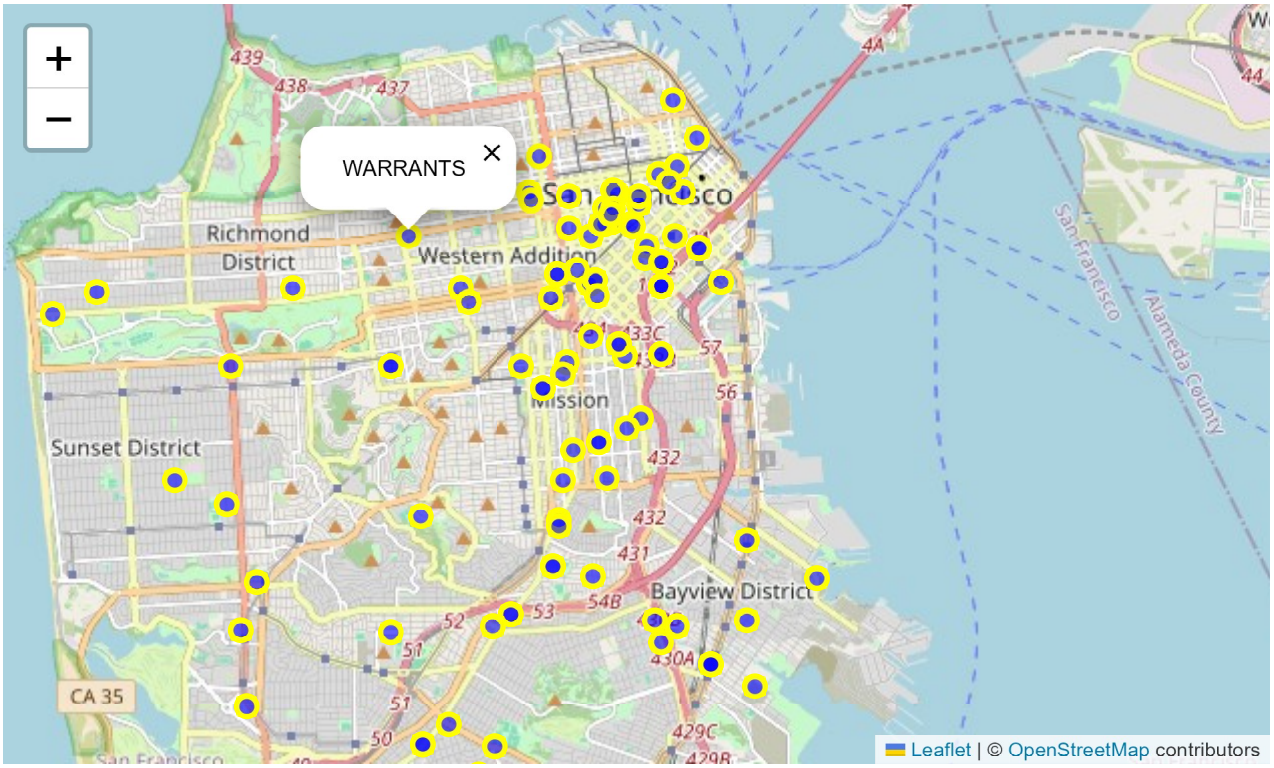


Result: Each marker now displays the crime category (e.g., "LARCENY/THEFT", "ASSAULT") **when clicked or hovered over.**

5.4 Simplifying: Inline Popups on Circle Markers

If the map gets too congested, I can **embed the popup text directly** into the circle markers instead of adding separate marker objects:

```
sanfran_map = folium.Map(location=[latitude, longitude], zoom_start=12)

for lat, lng, label in zip(df_incidents.Y, df_incidents.X,
df_incidents.Category):
    folium.vector_layers.CircleMarker(
        [lat, lng],
        radius=5,
        color='yellow',
        fill=True,
        popup=label, # Show text on click
        fill_color='blue',
        fill_opacity=0.6
        #tooltip=label   # <-- shows when hover
    ).add_to(sanfran_map)

sanfran_map
```

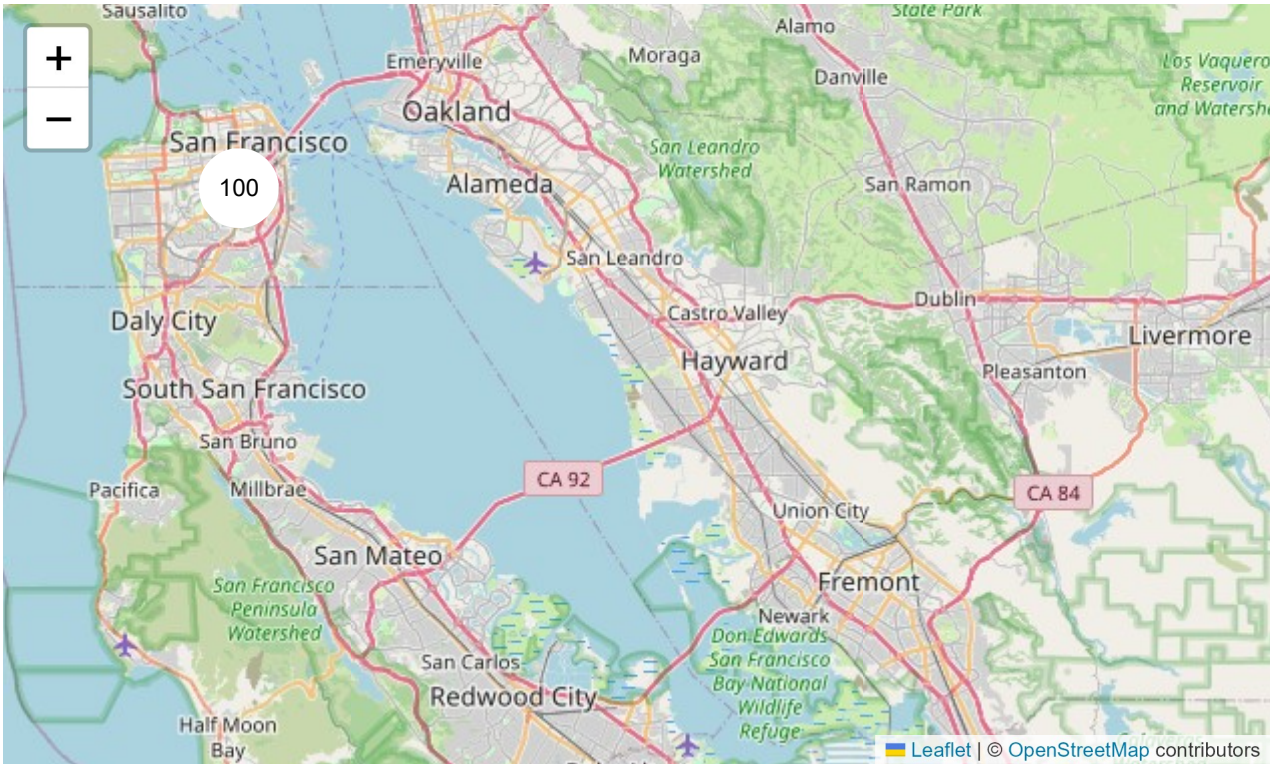


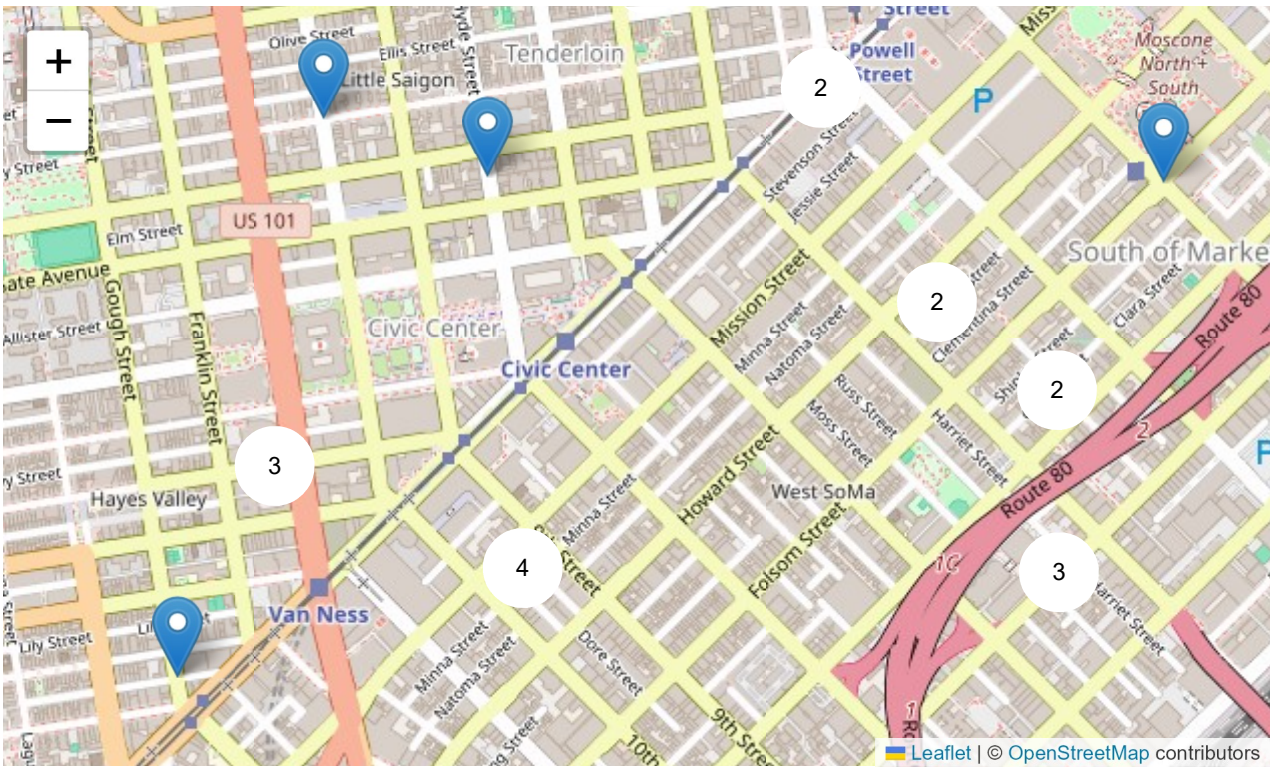
Result: A cleaner map with popup labels directly on the circle markers, reducing visual clutter.

5.5 Marker Clustering

The **most scalable** approach is to use **Marker Clusters**, which **group nearby markers into clusters** and break them apart as you zoom in:

```
from folium import plugins

# a clean copy of the map of San Francisco
sanfran_map = folium.Map(location=[latitude, longitude], zoom_start=12)

# instantiate a mark cluster object for the incidents in the dataframe
incidents = plugins.MarkerCluster().add_to(sanfran_map)

# loop through the dataframe and add each data point to the mark cluster
for lat, lng, label in zip(df_incidents.Y, df_incidents.X,
df_incidents.Category):
    folium.Marker(
        location=[lat, lng],
        icon=None,
        popup=label,
    ).add_to(incidents)

sanfran_map
```





Result: At full zoom-out, all 100 markers appear as a single cluster showing "100". As I **zoom in, clusters break down into smaller groups**, and at maximum zoom each marker appears individually with its crime-category popup.

6. Choropleth Maps

A choropleth map **shades geographic regions in proportion to a statistical variable** such as election votes - in this case, **total immigration to Canada** from each country.

6.1 Preparing the GeoJSON File

I need a GeoJSON file, **defining country boundaries**. I download a pre-prepared world countries GeoJSON:

```
!curl -o world_countries.json https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/world_countries.json
```

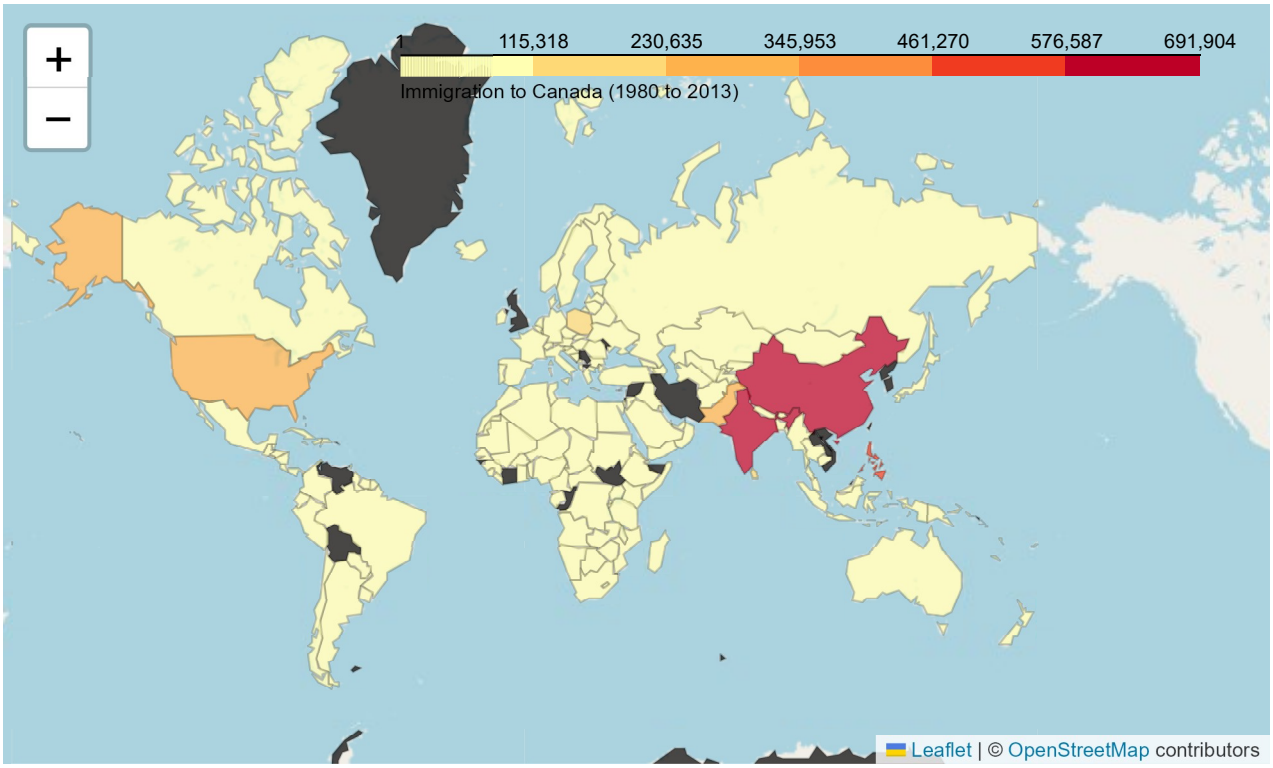
```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed

  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
  0     0    0     0    0     0      0      0 --:--:--  0:00:01 --:--:--     0
  0     0    0     0    0     0      0      0 --:--:--  0:00:01 --:--:--     0
  6  246k    6 15348    0     0   6119      0  0:00:41  0:00:02  0:00:39  6124
 71  246k   71  175k    0     0  51425      0  0:00:04  0:00:03  0:00:01 51460
100  246k  100  246k    0     0  68973      0  0:00:03  0:00:03 --:--:-- 69015
```

6.2 Building the Choropleth Map

I create a base world map and overlay the choropleth layer. The `choropleth()` method takes:

- `geo_data` - the GeoJSON file with country boundaries.
- `data` - the dataframe with immigration numbers.
- `columns` - which dataframe columns to use (country name and total immigrants).
- `key_on` - the key or variable in the GeoJSON file that contains the name of the variable of interest. To determine that, **open the GeoJSON file using any text editor and note the name of the key or variable** that contains the name of the countries. Note that this key is **case_sensitive**, so **pass exactly** as it exists in the GeoJSON file (here, **`feature.properties.name`**).
- `fill_color` - a ColorBrewer color scale (`YlOrRd` goes from yellow to red).

```
# a GeoJSON that defines the boundaries of all world countries
world_geo = r'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/
world_countries.json'

world_map = folium.Map(location=[0, 0], zoom_start=2)

folium.Choropleth(
    geo_data=world_geo,
    data=df_can,
    columns=['Country', 'Total'],
    key_on='feature.properties.name',
    fill_color='YlOrRd',
    fill_opacity=0.7,
    line_opacity=0.2,
    legend_name='Immigration to Canada (1980 to 2013)',
).add_to(world_map)

world_map
```



Result: A world choropleth map where each country is shaded from light yellow to deep red based on total immigrants to Canada between 1980 and 2013. A legend in the top-right corner shows the color gradient with corresponding immigration counts.

The map clearly reveals that **China**, **India**, and the **Philippines** have the highest immigration levels to Canada over the 33-year period, followed by Poland, Pakistan, and the United States. Countries with minimal immigration appear light yellow or remain unshaded if data is unavailable.

That's the full walkthrough. I set up Folium with datasets, built interactive crime maps of San Francisco using Circle and Cluster markers, and created a choropleth with GeoJSON to visualize global immigration to Canada - showing how Python makes geospatial data clear and interactive.

Interesting idea tip: Try experimenting with Choropleth maps for individual years or grouped decades.